Discover Electronics

RESEARCH Open Access

Comparative energy awareness and performance analysis of ARM and RISC-V SHA-256 hardware acceleration on the RP2350

Atul Ravi[1*], Sriram S.[1], B. Kailash[1] and V. Sarada[1]

*Correspondence:
Atul Ravi
ar4392@srmist.edu.in
[1]Department of Electronics and Communication Engineering, SRM Institute of Science and Technology, Kattankulathur, Chennai, Tamil Nadu, India

Abstract

SHA-256 is a fundamental hashing algorithm in embedded security, authentication, data integrity verification, and hardware random number generation in resource-constrained systems. This paper presents a quantitative benchmarking study of SHA-256 on the Raspberry Pi RP2350, a microcontroller which has integrated both ARM Cortex-M33 and RISC-V Hazard3 cores on a single die, enabling a direct same-silicon architectural comparison under identical conditions. Throughput, execution time, current draw, and energy per kilobyte were measured across software and hardware-accelerated modes with and without Direct Memory Access on both cores, and against a software baseline on the predecessor RP2040. Hardware acceleration yielded speedups of 14.05× and 13.58× over software for the ARM and RISC-V cores respectively, with energy per KB reduced from approximately 25 μJ/KB to under 1.79 μJ/KB in both cases, while current draw remained stable between 17 and 18 mA across all configurations, indicating that efficiency gains are attributable to reduced execution time rather than lower active current. A counter-intuitive observation emerges from the DMA analysis: despite the ARM ecosystem's significantly greater maturity in embedded bus integration, the ARM Cortex-M33 exhibits a larger DMA-induced throughput penalty (18.1%, factor of 1.221×) than the newer RISC-V Hazard3 core (13.6%, factor of 1.157×) when accessing the shared SHA-256 peripheral on identical silicon. To our knowledge, this is the first same-silicon comparison of ARM and RISC-V cryptographic acceleration with direct energy measurements, and the first published per-configuration energy characterization of the RP2350.

Keywords SHA-256, Hardware acceleration, RP2350, RP2040, RISC-V, ARM Cortex-M33, DMA, Energy efficiency, Embedded systems, Cryptography

1 Introduction

Raspberry Pi Ltd. (RPI) launched a range of chips as a part of their microcontroller line. The very first variant was the RP2040 with the ARM M0+ architecture. In 2024, the company announced an updated version of the chip, the RP2350, with a built-in SHA-256

© The Author(s) 2026. **Open Access** This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

accelerator, a better power supply layout, more features than its predecessor, and most importantly the possibility to use the built-in ARM Cortex-M33 cores or Hazard3 RISC-V cores. The multiple capabilities of the RP2350 made us wonder about its encoding efficiencies with and without its hardware accelerator.

The RP2040 has seen widespread use in embedded systems, and so has the RP2350. The RP2350's integrated accelerator is useful for purposes of cryptography and other random number generation processes. Our goal was to discern the difference in hardware performance when a complete hardware-based approach to hashing was implemented and compared to a fully software-based approach. The analysis made use of the available *NIST.FIPS.180-C* [1] test vectors to verify the operation of the library (in the case of the software approach), while the examples provided by Raspberry Pi were used as a standard and built upon for a thorough analysis of the hardware acceleration approach to hashing. From the analysis we attempt to draw parallels between throughput, cycles per byte, energy per kilobyte, and other hardware-related parameters such as current draw and binary size to identify the best approach for any hardware designer wanting to implement SHA-256 in their products/projects.

We also tested the effect of Direct Memory Access (DMA) on the hashing process to identify how efficient it would be if we have to use the CPU at the same time for other processes as well. This paper does not aim to go into detail of how the SHA-256 hashing is carried out at the die and register level but rather attempts to look at the capability from the embedded developer's perspective to identify which method is the most suitable.

Existing characterizations of SHA-256 acceleration on embedded platforms report throughput in isolation, either for ARM or RISC-V platforms separately, without per-operation energy measurements and without controlling for silicon, process, voltage, clock, and thermal conditions. The RP2350's dual-architecture design enables, for the first time in a commercially available part, direct comparison of ARM and RISC-V SHA-256 performance on identical silicon, eliminating confounds that limit all prior cross-architecture comparisons. This paper contributes (i) the first published per-configuration energy and throughput characterization for the RP2350, (ii) a same-silicon comparison of ARM Cortex-M33 and RISC-V Hazard3 cores accessing a shared SHA-256 accelerator, (iii) quantification of DMA-induced throughput penalty on both cores with an explanation of the underlying mechanism, and (iv) comparison against vendor benchmarks for ESP32 and STM32 platforms using the same cryptographic library to enable cross-platform reference.

2 Related work

SHA-256 hashing performance on ARM architectures has been characterized across architectures by Ledwaba et al. [2] benchmarking software-defined hashing performance on various STM32 processors using mbedTLS. It was established that power draw and execution time varied from a few milliseconds to a few seconds with corresponding energy draw varying from 102–2496 mJ. Notably, the Cortex-M3 exhibited both the highest execution time and energy consumption across all tested cores. This

was attributed to the differing clock frequencies and library optimisation conditions. No hardware acceleration results were reported.

Kietzmann et al. [3] extended this across multiple off-the-shelf IoT platforms using the RIOT OS crypto subsystem, finding that hardware-based cryptographic acceleration consistently outperforms software implementations by over 100%, and that hardware diversity and driver design significantly affect resource efficiency, though per-operation current draw was not reported.

On RISC-V, Stoffelen [4] established that the absence of native rotation instructions leads to reduced software-defined SHA-256 hashing relative to ARM architectures.

Vendor benchmarks from Oryx Embedded [5–7] and wolfSSL [8–10] provide throughput reference points for ESP32 (Xtensa, 240 MHz) and ESP32-C3/C6 (RISC-V, 160 MHz), reporting software SHA-256 throughput of 1,623-2,173 KB/s and hardware-accelerated throughput of 21,739-27,777 KB/s, but neither source reports current draw or energy per KB.

No published work benchmarks SHA-256 on the RP2350, nor compares ARM and RISC-V hashing on the same silicon with direct energy measurements. This paper addresses that gap.

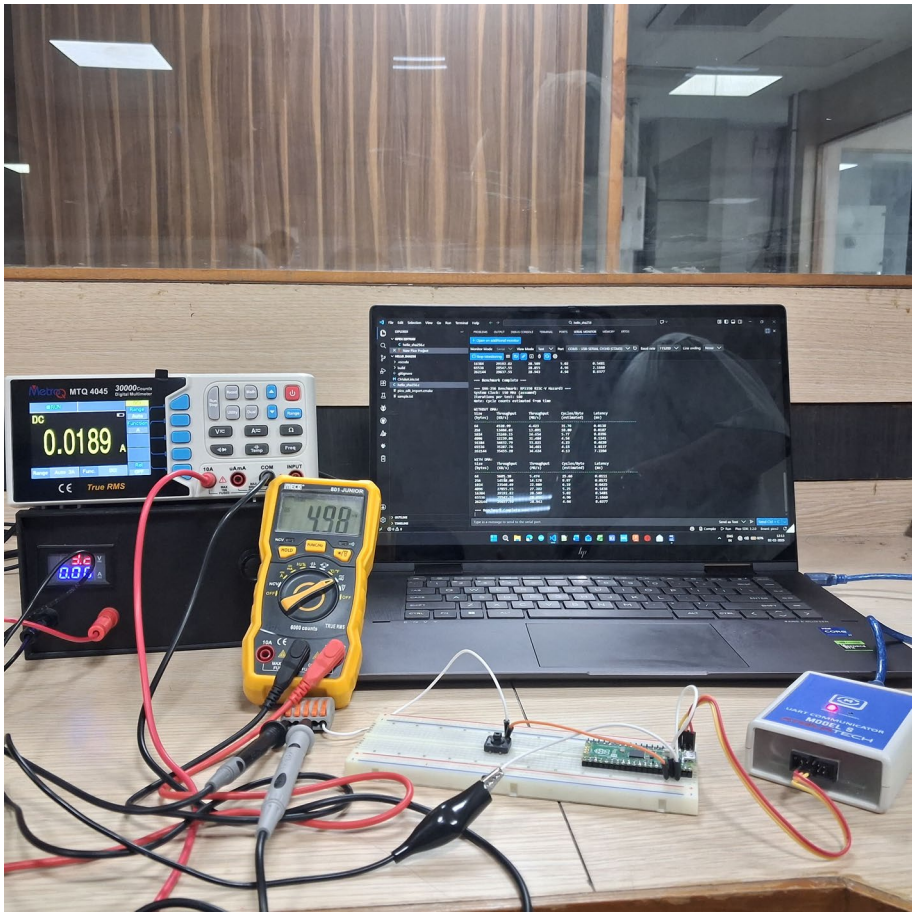


Fig. 1 Benchtop test setup with the RP2040, USB-UART Bridge and DMMs

3 Methodology

To obtain accurate data with the RP2350 and the RP2040 architectures, the RPI Pico and RPI Pico 2 development boards were utilized (see Fig. 1). By doing so, we were able to perform tests with multiple firmware variants and architectures without having to worry about variation in memory chips and LDO Regulators on the development boards.

We used the sample code provided by Raspberry Pi, `hello_SHA256`, as a base for our hardware-level tests of the built-in accelerator on the RP2350. This code was modified to show the throughputs with various data sizes, the latency, cycles per byte, and other parameters. For software analysis of the architectures, a library was developed as a wrapper of the CycloneCRYPTO SHA256 library used for their FPGAs, developed in C [11]. This wrapper ensured that the core `SHA256.h` [12] and `SHA256.c`[13] files are not modified to retain the main functionality of the code.

The output from the MCUs was recorded and observed with the Atrivatech UART Debugger tool, which is a USB-to-UART bridge running at 115200 baud-rate with no line ending, using the serial monitor built into Visual Studio Code (VS Code). All currents and voltages were measured with the MetroQ MTQ4045 multimeter, which has 4.5-digit accuracy. The "DCA" mode flag appearing in the per-trial data tables (Appendix A) is the MTQ4045's logging-format indicator for direct-current Ampere measurements and is preserved here for reproducibility with the same instrumentation. A constant voltage power supply at 3.3 V was used with the MTQ4045 multimeter and a MECO-805 Junior DMM in the mV range to track burden voltages (if any). A switch was placed in line with the positive wire to control the MCUs. Care was taken to ensure the voltage at the 3V3_out line was at a constant 3.296 V, which was provided as an input to the same pin with a constant voltage (CV) power supply. The code for all the tests was written in C++ and made use of the Raspberry Pi SDK version 2.2.0 with the plugin provided by Raspberry Pi on VS Code. All features were enabled, and STDIO support was enabled over UART, and the DAP probe debugger option was enabled (although not used). For RISC-V code, the RISC-V option was chosen. The chip stepping and clock speeds for both microcontrollers used in this study are summarized in Table 1.

For testing the accuracy of the software's SHA-256 hashing algorithm, standardized NIST vectors were used and verified against the hashed data, as shown in Fig. 2. Once verified, the code was updated to account for throughputs and other parameters.

3.1 Burden voltage

To ensure that the development boards receive maximum input voltage, the MECO-801 was placed in series with the MTQ4045 to identify the burden voltage. In the mA range, the multimeter showed a voltage of 102–114 mV. In the A range (where we achieved 4-digit accuracy), the DMM only had a burden voltage between 4.13 and 4.63 mV. Therefore, the A range was utilized for all tests.

Table 1 Chip stepping and clock speeds

Chip	Clock speed	Chip stepping
RP2040	133 MHz	RP2-B2
RP2350	150 MHz	RP2350A0A2

```
---- Opened the serial port COM11 ----
21:41:07:387 ->
21:41:07:387 -> === Test Vectors Verification ===
21:41:07:391 -> Test 1 (empty string):
21:41:07:391 ->   Expected: e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855
21:41:07:396 ->   Got:      e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855
21:41:07:402 ->   Result:   PASS
21:41:07:405 ->
21:41:07:405 -> Test 2 ("abc"):
21:41:07:405 ->   Expected: ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad
21:41:07:414 ->   Got:      ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad
21:41:07:419 ->   Result:   PASS
21:41:07:422 ->
21:41:07:422 -> Test 3 (56 chars):
21:41:07:425 ->   Expected: 248d6a61d20638b8e5c026930c3e6039a33ce45964ff2167f6ecedd419db06c1
21:41:07:432 ->   Got:      248d6a61d20638b8e5c026930c3e6039a33ce45964ff2167f6ecedd419db06c1
21:41:07:436 ->   Result:   PASS
21:41:07:439 ->
21:41:07:439 -> === Verification Summary ===
21:41:07:442 -> Tests passed: 3/3
21:41:07:445 -> SHA-256 implementation is CORRECT per NIST FIPS 180-4!
21:41:07:448 ->
21:41:07:448 -> === SHA-256 Benchmark: RP2040 Cortex-M0+ ===
21:41:07:453 -> Implementation: Software defined - C - Built-on Cyclone Crypto
21:41:07:459 -> Clock: 133 MHz
21:41:07:459 -> Iterations per test: 100
21:41:07:462 -> Note: Cycle counts estimated from time
21:41:07:464 ->
21:41:07:464 -> SOFTWARE SHA-256:
21:41:07:467 -> Size       Throughput      Throughput      Cycles/Byte     Latency
21:41:07:473 -> (bytes)    (KB/s)          (MB/s)          (estimated)     (ms)
21:41:07:481 -> ---------------------------------------------------------------------------
21:41:07:503 -> 64         467.81          0.457           277.64          0.1336
21:41:07:543 -> 256        755.10          0.737           172.01          0.3311
21:41:07:662 -> 1024       891.79          0.871           145.64          1.1213
21:41:08:098 -> 4096       934.05          0.912           139.05          4.2824
21:41:09:798 -> 16384      945.25          0.923           137.41          16.9268
21:41:16:558 -> 65536      948.09          0.926           136.99          67.5042
21:41:16:563 ->
21:41:16:563 -> === Benchmark Complete ===
```

Fig. 2 Output verified against 'abc' vector in NIST message digest [14] and throughputs for software-defined hashing on RP2040. Timestamps reflect a representative test run; measurement values are deterministic across runs

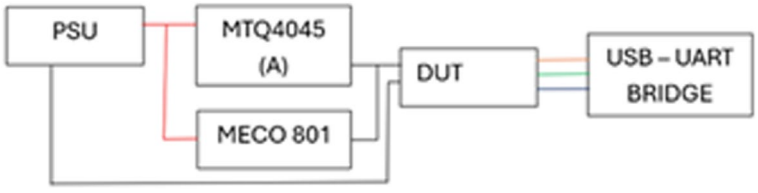


Fig. 3 Device under test layout (DUT). DUT here is the RP2350 and RP2040. Connections are made according to this layout

The assessments were segmented into five sections, with the device under test layout shown in Fig. 3. Out of the five, four pertained to the RP2350, while one was associated with the RP2040. The initial tests were conducted using the RP2350 and its hardware acceleration capabilities. The throughputs and corresponding data were copied from VS Code serial monitor and transferred to a separate text file. Simultaneously, as the code executed on the RP2350, currents were recorded and monitored with the software for the MTQ4045. The current data was exported as a CSV file from the software, including the time stamps and graphical representations. The timestamped current and throughput data for each variant were evaluated, and findings were recorded.

The RP2350 and RP2040 were reset 13 times using the inline switch while the chips were in sleep state. This was conducted to ascertain the throughput time, cycles per byte, and all other metrics along with their variance.

A clear analysis of which process demands the most current is necessary for low-power applications. For this, separate code was written for both architectures where the hashing process and memory access were run. This code split the processes with a 5-second gap between each to provide a distinct 'baseline'. From this, sleep currents and

process-specific currents were also recorded for further analysis. This data was used in the energy per KB calculations.

3.2 Energy per KB (µJ/KB)

The energy in joules is given by (1), where 3.296 is the input voltage, I the current in Amps (A), and T the time in milliseconds (ms). The energy obtained is then substituted in (2) to obtain the energy per kilobyte. Here 4096 is the number of bytes considered, and the power of 10^6 is to account for the kilobyte and microjoule unit conversions.

$$E(J) = 3.296 \times I \times \left(\frac{T}{1000}\right) \tag{1}$$

$$\mu J/KB = \frac{E(J)}{4096/1024} \times 10^6 \tag{2}$$

Current measurements reported here represent time-averaged draw across benchmark intervals captured by the MTQ4045 at its native sampling rate; per-operation current resolution at sub-millisecond execution times would require higher-bandwidth instrumentation and is not captured in this study.

4 Results and discussions

4.1 Hashing data analysis

Per-trial measurement data for all seven configurations (RP2040 software, RP2350 ARM software, RP2350 ARM hardware with and without DMA, RP2350 RISC-V software, RP2350 RISC-V hardware with and without DMA) across 13 repetitions each are provided in Appendix A. The aggregated mean values from these measurements are summarized in Table 2 and used throughout the subsequent analysis.

4.2 RP2350 and RP2040 hashing analysis

From the per-trial measurement data, we can condense the primary chip and architecture data as seen in Table 2. From the condensed data, we can see that the energy per kilobyte is lowest for the ARM architecture with hardware accelerator, and highest for the RP2040 with a totally software approach.

Consistent with the throughput penalty observed under DMA, both cores show modestly higher energy per KB in DMA configurations than in non-DMA hardware modes,

Table 2 Energy and performance comparison Of RP2040 and RP2350

Chip	Family	Mode	Binary size	Throughput (KB/s)	Avg current (A)	Execution time	Total energy (J)	Energy per KB (μJ/KB)
RP2350	ARM	DMA	23KB	27605	0.0180	0.1499	8.89E-06	2.22
RP2350	ARM	No DMA	23KB	33704	0.0175	0.1187	6.86E-06	1.72
RP2350	RISC-V	No DMA	31KB	32239	0.0174	0.1241	7.14E-06	1.78
RP2350	RISC-V	DMA	31KB	27854	0.0173	0.1436	8.19E-06	2.05
RP2350	ARM	Software	24KB	2398	0.0182	1.6673	9.99E-05	24.97
RP2350	RISC-V	Software	33KB	2372	0.0174	1.6857	9.68E-05	24.22
RP2040	ARM M0	Software	26KB	934	0.0232	4.2824	3.27E-04	81.84

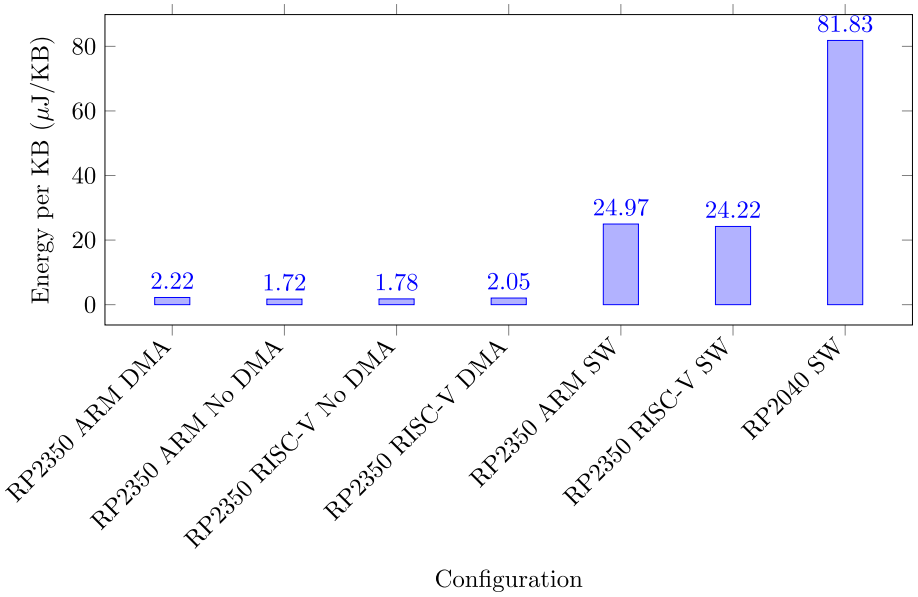


Fig. 4 Energy per KB (μJ/KB) across configurations

though both remain approximately 12× more efficient than their respective software baselines. Fig. 4 visualizes the energy per KB across all configurations.

The variance of the data has been ignored as it is well below 0.007% of the actual data hereby reducing the calculation complexity.

The decreased performance of the RP2040 in terms of energy can also be attributed to the internal power regulator layout. Part of the energy efficiency improvement on the RP2350 can be attributed to its on-die core voltage regulator architecture. The RP2040 uses an internal LDO to derive its core rail from the 3.3 V supply, while the RP2350 uses an internal switched-mode regulator for the same conversion, which is more efficient at the conversion ratio involved and accounts for the external inductor requirement on RP2350-based designs. Both Raspberry Pi Pico and Pico 2 development boards use external buck regulators for the 5 V to 3.3 V conversion at the board level; the distinction discussed here is at the die level, not the board level.

4.3 DMA and no DMA

The DMA analysis shows that memory-access overhead reduces throughput on both cores relative to non-DMA hardware-accelerated mode. RISC-V throughput drops from 32,239 KB/s to 27,854 KB/s, a 13.6% reduction (factor of 1.157×), while ARM Cortex-M33 throughput drops from 33,704 KB/s to 27,605 KB/s, an 18.1% reduction (factor of 1.221×).

The observation that DMA reduces rather than increases throughput merits explanation, as DMA is conventionally expected to accelerate transfers by offloading the CPU. In this measurement, the CPU is not the bottleneck: direct memory-mapped writes from the core feed the SHA-256 peripheral at or near its operational throughput limit. Introducing DMA in this path adds channel configuration, descriptor processing, and peripheral handshake overhead without offsetting any CPU-bound delay, because no such

delay exists in the non-DMA case. The result is net negative throughput impact. DMA remains advantageous in workloads where the CPU has competing tasks and would otherwise stall the peripheral, but for sustained dedicated hashing, the direct path is faster.

Counter to the expectation that the more mature ARM ecosystem would exhibit lower DMA overhead than the comparatively new Hazard3 RISC-V core, the ARM core in fact shows the larger DMA-induced penalty on identical silicon. The throughput variance across 13 trials in each configuration is below 14.8 $(KB/s)^2$ for ARM hardware modes and below 3.9 $(KB/s)^2$ for RISC-V hardware modes (Appendix A, Tables 8, 9, 11, 12), corresponding to coefficients of variation below 0.05% in all cases. The observed 5.6 percentage-point difference between the ARM and RISC-V DMA-induced penalties (18.1% versus 13.6%) corresponds to a throughput differential of approximately 6,100 KB/s on a baseline of roughly 33,000 KB/s, three orders of magnitude larger than the standard deviation of either configuration. The asymmetry is therefore robust against trial-to-trial measurement variation. This points to a measurable asymmetry in how each core complex interacts with the SHA-256 peripheral, the mechanism of which is not localized by the present measurements. Characterization across payload sizes and register-level analysis of the bus interactions between each core and the peripheral are left for future work. For maximum SHA-256 throughput, non-DMA hardware mode is preferred on both cores; DMA remains useful in scenarios where CPU availability for concurrent tasks outweighs the throughput cost. Fig. 5 shows the measured current draw on the RP2350-ARM core in DMA and non-DMA hardware modes.

Despite the throughput penalty, the DMA configurations still achieve energy per KB values of 2.22 µJ/KB (ARM) and 2.05 µJ/KB (RISC-V), approximately 12× improvements over the respective software baselines.

The DMA-enabled measurements presented in this study use the Raspberry Pi SDK v2.2.0 `pico_sha256` high-level API with the `use_dma` parameter set to `true`, invoked from the same `hello_sha256` sample referenced in Sect. 3. This API exposes DMA usage to the application as a single boolean flag and handles all underlying configuration

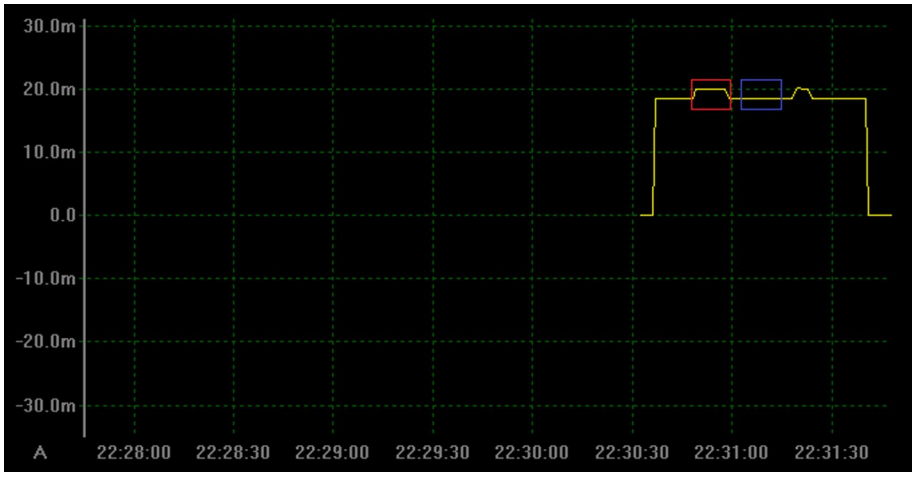


Fig. 5 Highlighted in red, dma current draw and highlighted in blue, current draw without DMA on the RP2350-ARM as obtained with the MTQ4045 software

Table 3 Throughput comparison with cyclone CRYPTO - Oryx embedded data

Chip	Family	Clock (MHz)	Mode	Throughput (KB/s)
STM32H753ZI [5]	ARM Cortex-M7	400	Hardware (HASH peripheral)	105263
RP2350	RISC-V	150	Hardware (DMA)	27855
RP2350	ARM Cortex-M33	150	Hardware (DMA)	27606
ESP32 [6]	Xtensa LX6	240	Hardware (SHA accelerator)	27777
ESP32-C3 [7]	RISC-V	160	Hardware (SHA accelerator)	21739
STM32H753ZI [5]	ARM Cortex-M7	400	Software	11061
RP2350	ARM Cortex-M33	150	Software	2399
RP2350	RISC-V	150	Software	2373
ESP32 [6]	Xtensa LX6	240	Software	2173
ESP32-C3 [7]	RISC-V	160	Software	1623
RP2040	ARM-Cortex - M0+	133	Software	934

Table 4 Throughput comparison with WolfSSL data

Chip	Core	Clock (MHz)	Mode	SHA-256 throughput (KB/s)	Block size (bytes)
RP2350	RISC-V	150	Hardware (DMA)	27854	4096
RP2350	ARM Cortex - M33	150	Hardware (DMA)	27606	4096
ESP32-WROOM-32 [8]	Xtensa LX6	240	Hardware (SHA accelerator)	26598	1024
ESP32-C6 [10]	RISC-V	160	Hardware (SHA accelerator)	12612	1024
RP2350	ARM Cortex - M33	150	Software	2399	4096
RP2350	RISC-V	150	Software	2373	4096
ESP32-WROOM-32 [8]	Xtensa LX6	240	Software	1790	1024
STM32F2 [16]	ARM Cortex-M3	120	Software	1484	1024
ESP32-C3 [9]	RISC-V	160	Software	1450	1024
ESP32-C6 [10]	RISC-V	160	Software	1450	1024
RP2040	ARM-Cortex - M0+	133	Software	934	4096

internally, including channel allocation, source and destination buffer routing, transfer width, and any descriptor-level byte-swap settings. The benchmark code does not configure these directly. The input buffer for each benchmark iteration is allocated dynamically via `malloc` and populated immediately before the SHA-256 call, placing the data in SRAM rather than flash, so flash fetch latency does not contribute to the measured DMA timing. The SHA-256 peripheral endianness is set via the `SHA256_BIG_ENDIAN` flag passed to `pico_sha256_start_blocking`; whether the SDK additionally enables the DMA controller's hardware BSWAP feature in its descriptor configuration is internal to the SDK and not exposed at the API level. The throughput characterization reported here therefore reflects SHA-256 performance as achieved through the documented high-level SDK usage pattern, which corresponds to what most embedded developers will encounter when adapting the `hello_sha256` example. Further optimization through direct DMA register configuration, manual SRAM bank partitioning, or descriptor-level BSWAP enablement may reduce the observed overhead and is identified as a direction for future work alongside the payload size and register-level characterization noted earlier. The throughput pattern observed in this study, namely that DMA-enabled SHA-256 transfers run slower than non-DMA transfers under sustained

Table 5 Speed-Up ratio comparison between RP2350 and RP2040

Chip	Core	SW throughput (KB/s)	HW throughput (KB/s)	Speed-up ratio	Comparison type	Notes
RP2350	ARM Cortex-M33	2399	27605	11.51	HW DMA vs SW	DMA affects ARM throughput
RP2350	ARM Cortex-M33	2399	33704	14.05	HW No DMA vs SW	Best ARM configuration
RP2350	RISC-V Hazard3	2373	32239	13.58	HW No DMA vs SW	Best RISC-V configuration
RP2350	RISC-V Hazard3	2373	27855	11.74	HW DMA vs SW	DMA reduces throughput on RISC-V
RP2040 vs RP2350	ARM Cortex-M0+ vs ARM Cortex-M33	934	2399	2.57	Generational SW comparison	SW only no hardware accelerator on RP2040

hashing, is consistent with an independent observation reported by a Raspberry Pi forum contributor in September 2025 [15] using the same `hello_sha256` sample on a Pico 2 W in RISC-V mode, where DMA added approximately 5 ms of overhead per megabyte hashed relative to non DMA operation. This independent observation supports the conclusion that the behavior is characteristic of the standard SDK usage pattern rather than an artifact of our specific test conditions.

4.4 Comparison with industry benchmarks

Oryx Embedded and wolfSSL have conducted multiple benchmarks with related hardware and architectures.

The data in table 3 shows the RP2350 exhibits higher throughput than the ESP32 in both hardware and software modes, despite the ESP32's higher clock speed. From this we can understand that the clock speed does not necessarily affect the performance of hashing. The hashing performance is largely dependent on the architecture of the hashing peripheral. We must also note that the data size (block size) that was hashed has not been provided. A further throughput comparison drawing on wolfSSL benchmarks is provided in Table 4, where block sizes are explicitly noted.

4.5 Speedup ratio

Across the RP2350 and RP2040, the speedup ratio between software and hardware can be characterised by Table 5.

The speedup ratios in Table 5 show that DMA reduces throughput on both cores relative to their non-DMA hardware-accelerated modes. The ARM Cortex-M33 speedup drops from $14.05\times$ (no DMA) to $11.51\times$ (DMA), a penalty factor of $1.221\times$, while the RISC-V Hazard3 speedup drops from $13.58\times$ (no DMA) to $11.74\times$ (DMA), a penalty factor of $1.157\times$. The ARM core exhibits a marginally larger DMA-induced penalty than the RISC-V core under these conditions, indicating that DMA introduces measurable interfacing overhead on both ISAs when accessing the shared SHA-256 peripheral.

5 Conclusion

This paper has presented an empirical characterisation of SHA-256 hashing performance on the Raspberry Pi RP2350, a dual-architecture microcontroller integrating both ARM Cortex-M33 and RISC-V (Hazard3) cores on a single die. Throughput, execution time, current draw, and energy per KB were measured across software and hardware-accelerated modes, with and without DMA, providing a complete energy efficiency profile unavailable in existing vendor benchmarks for comparable platforms.

Hardware acceleration had speedups of 14.05× and 13.58× over the software defined hashing with the ARM and RISC-V cores respectively in optimal configurations. The RP2350 ARM Cortex-M33 with software defined hashing alone achieved 2.57× the throughput of the RP2040 Cortex-M0+, showing a generational improvement independent of hardware acceleration.

DMA reduces SHA-256 throughput on both cores relative to non-DMA hardware-accelerated mode. The ARM Cortex-M33 experiences a 1.221× penalty (33,704 KB/s to 27,605 KB/s), while the RISC-V Hazard3 experiences a smaller 1.157× penalty (32,239 KB/s to 27,854 KB/s). Counter to the expectation that the more mature ARM ecosystem would exhibit lower DMA overhead than the newer Hazard3 core, the ARM core shows the larger DMA-induced penalty on identical silicon. This points to a measurable asymmetry in how each core complex interacts with the shared SHA-256 peripheral; the underlying mechanism is not localized by the present measurements and requires payload-size sweeps and register-level analysis to characterize in future work.

Energy measurements have shown that the hardware acceleration reduces energy per KB from 24.97 µJ/KB to 1.72–2.22 µJ/KB with ARM, and from 24.22 µJ/KB to 1.78–2.05 µJ/KB on RISC-V, representing approximately 12× energy efficiency gains in both cases. Current draw remained consistent across all configurations at approximately 17 to 18 mA, confirming that energy savings are realised due to reduced execution time rather than lower active current.

Based on these findings, further study is necessary to characterize the DMA throughput penalty across a range of payload sizes to localize its mechanism at the peripheral interface level, and to perform register-level analysis of the bus interactions between each core and the shared SHA-256 accelerator. Extension of the methodology to other cryptographic operations supported by the RP2350's accelerator (such as authenticated encryption modes) would also strengthen the per-configuration energy characterization presented in this paper.

Appendix A Per-trial measurement data

The following tables present the per-trial measurements (13 trials each) used to compute the aggregated means in Table 2. These data are provided for transparency regarding measurement variance.

See Table 6, 7, 8, 9, 10, 11 and 12.

Table 6 Hashing data comparison over 13 attempts - RP2040

Index	Time	Mode	Data (A)	Throughput (KB/s)	Throughput (MB/s)	Cycles/byte (estimated)	Latency (ms)
1146	2026-01-2	DCA	0.0249	934.05	0.912	139.05	4.2824
1160	2026-01-2	DCA	0.0227	934.05	0.912	139.05	4.2824
1175	2026-01-2	DCA	0.0228	934.05	0.912	139.05	4.2824
1190	2026-01-2	DCA	0.0227	934.05	0.912	139.05	4.2824
1203	2026-01-2	DCA	0.0228	934.05	0.912	139.05	4.2824
1216	2026-01-2	DCA	0.0229	934.05	0.912	139.05	4.2824
1230	2026-01-2	DCA	0.0228	934.05	0.912	139.05	4.2824
1248	2026-01-2	DCA	0.0230	934.05	0.912	139.05	4.2824
1267	2026-01-2	DCA	0.0229	934.05	0.912	139.05	4.2824
1288	2026-01-2	DCA	0.0253	934.05	0.912	139.05	4.2824
1311	2026-01-2	DCA	0.0228	934.05	0.912	139.05	4.2824
1328	2026-01-2	DCA	0.0229	934.05	0.912	139.05	4.2824
1344	2026-01-2	DCA	0.0230	934.05	0.912	139.05	4.2824
1361	2026-01-2	DCA	0.0228	934.05	0.912	139.05	4.2824
	Mean		0.023192308	934.05	0.912	139.05	4.2824
	Variance		0.00000732436	0.00000000000	0.00000000000	0.00000000000	0.00000000000
	Percentage		0.003158098397	0.00000000000	0.00000000000	0.00000000000	0.00000000000

Table 7 Hashing data comparison over 13 attempts - RP2350 - ARM -software

Index	Time	Mode	Data (A)	Throughput (KB/S)	Throughput (MB/S)	Cycles/byte estimated	Latency (ms)
693	2026-01-2	DCA	0.0179	2399	2.343	61.06	1.6674
706	2026-01-2	DCA	0.0183	2399.03	2.343	61.06	1.6673
723	2026-01-2	DCA	0.0180	2398.97	2.343	61.06	1.6674
739	2026-01-2	DCA	0.0182	2399	2.343	61.06	1.6674
754	2026-01-2	DCA	0.0182	2399	2.343	61.06	1.6674
767	2026-01-2	DCA	0.0182	2399.05	2.343	61.06	1.6673
781	2026-01-2	DCA	0.0183	2398.99	2.343	61.06	1.6674
800	2026-01-2	DCA	0.0183	2399.02	2.343	61.06	1.6674
818	2026-01-2	DCA	0.0182	2398.99	2.343	61.06	1.6674
834	2026-01-2	DCA	0.0182	2398.98	2.343	61.06	1.6674
852	2026-01-2	DCA	0.0183	2398.99	2.343	61.06	1.6674
872	2026-01-2	DCA	0.0182	2398.94	2.343	61.06	1.6674
884	2026-01-2	DCA	0.0182	2399	2.343	61.06	1.6674
	Mean		0.018176923	2398.993077	2.343	61.06	1.667384615
	Variance		0.000000016923	0.001073076923	0.000000000000	0.000000000000	0.000000001410
	Percentage		0.000093101989	0.000044730305	0.000000000000	0.000000000000	0.000000084579

Table 8 Hashing data comparison over 13 attempts - RP2350 - ARM - Hardware - Without DMA

Index	Time	Mode	Data (A)	Throughput (KB/S)	Throughput (MB/S)	Cycles/byte estimated	Latency (ms)
1414	2026-01-2	DCA	0.018	33709.76	32.92	4.34	0.1187
1423	2026-01-2	DCA	0.0181	33701.24	32.911	4.34	0.1187
1429	2026-01-2	DCA	0.0179	33701.24	32.911	4.34	0.1187
1437	2026-01-2	DCA	0.0178	33709.76	32.92	4.34	0.1187
1446	2026-01-2	DCA	0.018	33706.92	32.917	4.34	0.1187
1456	2026-01-2	DCA	0.018	33704.08	32.914	4.34	0.1187
1467	2026-01-2	DCA	0.0178	33709.76	32.92	4.34	0.1187
1477	2026-01-2	DCA	0.018	33701.24	32.911	4.34	0.1187
1488	2026-01-2	DCA	0.018	33704.08	32.914	4.34	0.1187
1500	2026-01-2	DCA	0.0178	33701.24	32.911	4.34	0.1187
1509	2026-01-2	DCA	0.018	33704.08	32.914	4.34	0.1187
1515	2026-01-2	DCA	0.0127	33701.24	32.911	4.34	0.1187
1523	2026-01-2	DCA	0.0179	33698.40	32.909	4.34	0.1187
	Mean		0.017538462	33704.08	32.91407692	4.34	0.1187
	Variance		0.000002122564	14.786933333342	0.000015576923	0.0000000000	0.0000000000
	Percentage		0.012102339181	0.043872828848	0.000047326021	0.0000000000	0.0000000000

Table 9 Hashing data comparison over 13 attempts - RP2350 - ARM - Hardware - With DMA

Index	Time	Mode	Data (A)	Throughput (KB/S)	Throughput (MB/S)	Cycles/byte estimated	Latency (ms)
1415	2026-01-2	DCA	0.0181	27605.24	26.958	5.3	0.1449
1424	2026-01-2	DCA	0.0180	27603.34	26.956	5.3	0.1449
1430	2026-01-2	DCA	0.0180	27607.15	26.960	5.3	0.1449
1438	2026-01-2	DCA	0.0180	27605.24	26.958	5.3	0.1449
1447	2026-01-2	DCA	0.0180	27605.24	26.958	5.3	0.1449
1457	2026-01-2	DCA	0.0180	27605.24	26.958	5.3	0.1449
1468	2026-01-2	DCA	0.0181	27607.15	26.960	5.3	0.1449
1478	2026-01-2	DCA	0.0179	27605.24	26.958	5.3	0.1449
1489	2026-01-2	DCA	0.0179	27605.24	26.958	5.3	0.1449
1501	2026-01-2	DCA	0.0180	27607.15	26.960	5.3	0.1449
1510	2026-01-2	DCA	0.0180	27603.34	26.956	5.3	0.1449
1516	2026-01-2	DCA	0.0179	27607.15	26.960	5.3	0.1449
1524	2026-01-2	DCA	0.0179	27607.15	26.960	5.3	0.1449
	Mean		0.017984615	27605.68231	26.95846154	5.3	0.1449
	Variance		0.000000004744	1.909769230770	0.000002102564	0.000000000000	0.000000000000
	Percentage		0.000026375820	0.006918029446	0.000007799273	0.000000000000	0.000000000000

Table 10 Hashing data comparison over 13 attempts - RP2350 – RISC-V - Software

Index	Time	Mode	Data (A)	Throughput (KB/S)	Throughput (MB/S)	Cycles/byte estimated	Latency (ms)
968	2026-01-2	DCA	0.0175	2372.8	2.317	61.73	1.6858
977	2026-01-2	DCA	0.0174	2372.87	2.317	61.73	1.6857
987	2026-01-2	DCA	0.0174	2372.83	2.317	61.73	1.6858
997	2026-01-2	DCA	0.0175	2372.83	2.317	61.73	1.6858
1008	2026-01-2	DCA	0.0175	2372.83	2.317	61.73	1.6858
1019	2026-01-2	DCA	0.0173	2372.84	2.317	61.73	1.6858
1030	2026-01-2	DCA	0.0176	2372.80	2.317	61.73	1.6858
1041	2026-01-2	DCA	0.0174	2372.87	2.317	61.73	1.6857
1055	2026-01-2	DCA	0.0174	2372.79	2.317	61.74	1.6858
1076	2026-01-2	DCA	0.0175	2372.83	2.317	61.73	1.6858
1085	2026-01-2	DCA	0.0173	2372.84	2.317	61.73	1.6857
1099	2026-01-2	DCA	0.0174	2372.86	2.317	61.73	1.6857
1112	2026-01-2	DCA	0.0175	2372.79	2.317	61.74	1.6858
	Mean		0.017438462	2372.829231	2.317	61.73153846	1.685761538
	Variance		0.000000007564	0.000774358974	0.000000000000	0.000014102564	0.000000025564
	Percentage		0.000043375974	0.000032634417	0.000000000000	0.000022844991	0.000000152104

Table 11 Hashing data comparison over 13 attempts - RP2350 - RISC - Hardware - Without DMA

Index	Time	Mode	Data (A)	Throughput (KB/S)	Throughput (MB/S)	Cycles/byte estimated	Latency (ms)
1744	2026-01-2	DCA	0.0174	32239.86	31.484	4.54	0.1241
1754	2026-01-2	DCA	0.0174	32327.27	31.482	4.54	0.1241
1762	2026-01-2	DCA	0.0174	32239.86	31.484	4.54	0.1241
1772	2026-01-2	DCA	0.0175	32239.86	31.484	4.54	0.1241
1783	2026-01-2	DCA	0.0173	32239.86	31.484	4.54	0.1241
1790	2026-01-2	DCA	0.0175	32327.27	31.482	4.54	0.1241
1798	2026-01-2	DCA	0.0174	32327.27	31.482	4.54	0.1241
1806	2026-01-2	DCA	0.0174	32239.86	31.484	4.54	0.1241
1815	2026-01-2	DCA	0.0176	32327.27	31.482	4.54	0.1241
1823	2026-01-2	DCA	0.0175	32242.46	31.487	4.54	0.1241
1834	2026-01-2	DCA	0.0175	32239.86	31.484	4.54	0.1241
1841	2026-01-2	DCA	0.0175	32242.46	31.487	4.54	0.1241
1850	2026-01-2	DCA	0.0174	32242.46	31.487	4.54	0.1241
	Mean		0.017446154	32239.66308	31.48407692	4.54	0.1241
	Variance		0.000000006026	3.884023076921	0.000003576923	0.000000000000	0.0000000000
	Percentage		0.000034538507	0.012047343881	0.000011361054	0.000000000000	0.0000000000

Table 12 Hashing data comparison over 13 attempts - RP2350 - RISC - Hardware - With DMA

Index	Time	Mode	Data (A)	Throughput (KB/s)	Throughput (MB/s)	Cycles/byte estimated	Latency (ms)
1745	2026-01-2	DCA	0.0175	27855.15	27.202	5.25	0.1436
1755	2026-01-2	DCA	0.0174	27855.15	27.202	5.25	0.1436
1763	2026-01-2	DCA	0.0171	27855.15	27.202	5.25	0.1436
1773	2026-01-2	DCA	0.0174	27855.15	27.202	5.25	0.1436
1784	2026-01-2	DCA	0.0172	27853.21	27.2000	5.25	0.1436
1791	2026-01-2	DCA	0.0173	27855.15	27.202	5.25	0.1436
1799	2026-01-2	DCA	0.0171	27855.15	27.202	5.25	0.1436
1807	2026-01-2	DCA	0.0174	27855.15	27.202	5.25	0.1436
1816	2026-01-2	DCA	0.0171	27855.15	27.202	5.25	0.1436
1824	2026-01-2	DCA	0.0171	27855.15	27.202	5.25	0.1436
1835	2026-01-2	DCA	0.0176	27853.21	27.2000	5.25	0.1436
1842	2026-01-2	DCA	0.0174	27853.21	27.2000	5.25	0.1436
1852	2026-01-2	DCA	0.0171	27855.15	27.202	5.25	0.1436
	Mean		0.017307692	27854.70231	27.20153846	5.25	0.1436
	Variance		0.000000029103	0.723769230771	0.000000769231	0.000000000000	0.000000000000
	Percentage		0.000168148148	0.002598373599	0.000002827894	0.000000000000	0.000000000000

Acknowledgements
The authors would like to declare the use of Claude AI by Anthropic in the development of the helper library functions for the Software hashing with RP2350 and in improving the layout of presented data in code used for throughput identification.

Author contributions
The corrosponding author A.R., carried out the formal research, analysed the data and drafted the manuscript. S.S. carried out data collection, data representation and analysis with A.R., B.K. formatted and revised the manuscript, Dr. V.S. supervised the research and proofread the manuscript. All authors have read and approved the final manuscript.

Funding
The authors declare that no funding has been received for the development of this research work.

Data availability
Datasets shall be made available at reasonable request by the reader(s).

Declarations

Ethics approval and consent to participate
Not applicable. This research does not involve human participants, human data, or animal subjects. Accepted principles of ethical and professional conduct have been followed throughout. This research does not involve human participants.

Consent for publication
Not applicable. This research does not involve human participants.

Competing interests
The corresponding author, AR, discloses a directorial role at Atrivatech Private Limited, a company whose USB-to-UART converter was used solely as a data acquisition interface in this study. This instrument was selected on the basis of technical suitability and availability, and played no role in the design, analysis, or interpretation of results. No financial benefit to Atrivatech Private Limited arises from the findings of this work. All remaining authors declare no competing interests.

Received: 19 March 2026 / Accepted: 6 July 2026
Published online: 27 July 2026

References

1. National Institute of Standards and Technology: Secure hash standard (SHS). Technical Report FIPS PUB 180-4, National Institute of Standards and Technology (2015). https://doi.org/10.6028/nist.fips.180-4
2. Ledwaba LPI, Hancke GP, Venter HS, Isaac SJ. Performance costs of software cryptography in securing new-generation internet of energy endpoint devices. IEEE Access. 2018;6:9303–23. https://doi.org/10.1109/ACCESS.2018.2793301.
3. Kietzmann P, Boeckmann L, Lanzieri L, Schmidt TC, Wählisch M. A performance study of crypto-hardware in the low-end IoT. In: Proceedings of the 18th International Conference on Embedded Wireless Systems and Networks (EWSN). New York, NY, USA: ACM; 2021.
4. Stoffelen K. Efficient cryptography on the RISC-V architecture. In: Schwabe P, Thériault N, editors. Progress in Cryptology – LATINCRYPT 2019. Cham: Springer; 2019. p. 323–40. https://doi.org/10.1007/978-3-030-30530-7_16.
5. Oryx Embedded: CycloneCRYPTO benchmark – STM32H7. https://www.oryx-embedded.com/benchmark/st/crypto-stm32h7.html. Accessed: Feb. 04, 2026
6. Oryx Embedded: CycloneCRYPTO benchmark – ESP32. https://www.oryx-embedded.com/benchmark/espressif/crypto-esp32.html. Accessed: Feb. 04, 2026
7. Oryx Embedded: CycloneCRYPTO benchmark – ESP32-C3. https://www.oryx-embedded.com/benchmark/espressif/crypto-esp32-c3.html. Accessed: Feb. 04, 2026
8. wolfSSL: wolfSSL on the Espressif ESP32. https://www.wolfssl.com/docs/espressif/. Accessed: Feb. 04, 2026
9. wolfSSL: wolfSSL on the Espressif ESP32-C3. https://www.wolfssl.com/wolfssl-on-the-espressif-esp32-c3-risc-v/. Accessed: Feb. 04, 2026
10. wolfSSL: Espressif RISC-V hardware accelerated cryptographic functions. https://www.wolfssl.com/espressif-risc-v-hardware-accelerated-cryptographic-functions-up-to-1000-faster-than-software/. Accessed: Feb. 04, 2026
11. Oryx Embedded: CycloneCRYPTO: Cryptographic library. https://github.com/Oryx-Embedded/CycloneCRYPTO. Accessed: Feb. 04, 2026
12. Oryx Embedded: sha256.h – SHA-256 source code. https://www.oryx-embedded.com/doc/sha256_8h.html. Accessed: Feb. 04, 2026
13. Oryx Embedded: sha256.c – SHA-256 source code. https://www.oryx-embedded.com/doc/sha256_8c_source.html. Accessed: Feb. 04, 2026
14. National Institute of Standards and Technology: Secure hash algorithm – message digest length = 256. https://csrc.nist.gov/csrc/media/projects/cryptographic-standards-and-guidelines/documents/examples/sha256.pdf. Accessed: Feb. 04, 2026
15. jancumps: SHA256 example on Pico2 RISCV takes longer with DMA. Raspberry Pi Forums (2025). https://forums.raspberrypi.com/viewtopic.php?p=2335928
16. wolfSSL: STM32 and wolfSSL hardware crypto and RNG support. https://www.wolfssl.com/stm32-and-wolfssl-hardware-crypto-and-rng-support/. Accessed: Feb. 04, 2026

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.